

Task: Use Claude Code to fill task-decomposition.md with a 6-8 task breakdown.
Goal: Produce only the required deliverable and verify it against the success criteria.

Claude Code can read/edit your repo and run commands, and @file references can attach files to prompts. ([Claude API Docs](#)) ([Claude API Docs](#))

Essential files not provided

I CANNOT CONFIRM the exact project paths without these:

- 1. Approved Task Comments specification
- 2. Full technical plan
- 3. Lesson principles
- 4. Actual repo structure

You provided enough summary to draft the breakdown, but Claude Code should inspect the repo for real file paths.

Step-by-step

1. Open Claude Code

From repo root:

claude

2. Ask Claude Code to inspect only

Paste:

Read @task-decomposition.md and inspect the repo structure. Identify likely files for comments model, repo, service, schemas, API routes, and tests. Do not edit yet. Also list any missing essential inputs.

Check that it names real files. If it says the spec, full plan, or lesson is missing, continue using the provided summary.

3. Ask Claude Code to write the deliverable

Paste:

Update @task-decomposition.md using the provided Task Comments summary. Create 6-8 atomic tasks across 3-4 phases. Each task must modify max 3 files, take 30-90 min, include 4-6 checkbox acceptance criteria, dependencies by task ID, handoff, dependency graph, parallel opportunities, execution strategy, file tree, and validation checklist. Necessary only.

4. Ask Claude Code to self-check

Paste:

Check @task-decomposition.md against every success criterion. Report only failures. Do not edit.

5. Fix only failures

If failures appear, paste:

Fix only the reported failures in @task-decomposition.md. Do not add extra content.

6. Final verification

Paste:

Verify: 6-8 tasks, 3-4 phases, each task <=3 files, 30-90 min, 4-6 checkboxes, dependencies clear, foundation to logic to API, at least one parallel opportunity. Report pass/fail only.

Expected final structure

Your completed file should contain:

Phase 1: Foundation
T001 Comment model
T002 Comment schemas

Phase 2: Data Access and Logic
T003 Comment repository
T004 Repository tests
T005 Comment service
T006 Service tests

Phase 3: API and Integration
T007 API endpoints
T008 API integration tests

Parallel opportunity to include:

After T001, T002 and T003 can proceed partly in parallel.
After T003 and T005, repository tests and service tests can proceed in parallel.

Dependency order:

T001 -> T003 -> T005 -> T007 -> T008
T002 -----> T005 -> T007
T003 -> T004
T005 -> T006